Project 2 - Classification Of Facial Images

Team 12: Steffi Dorothy & Neha Bathuri

October 22, 2024

Purpose

- Understand the dataset
- Analyse the data
- Training and evaluating machine learning & deep learning models for classification particularly focused on facial recognition , attribute analysis and image generation.

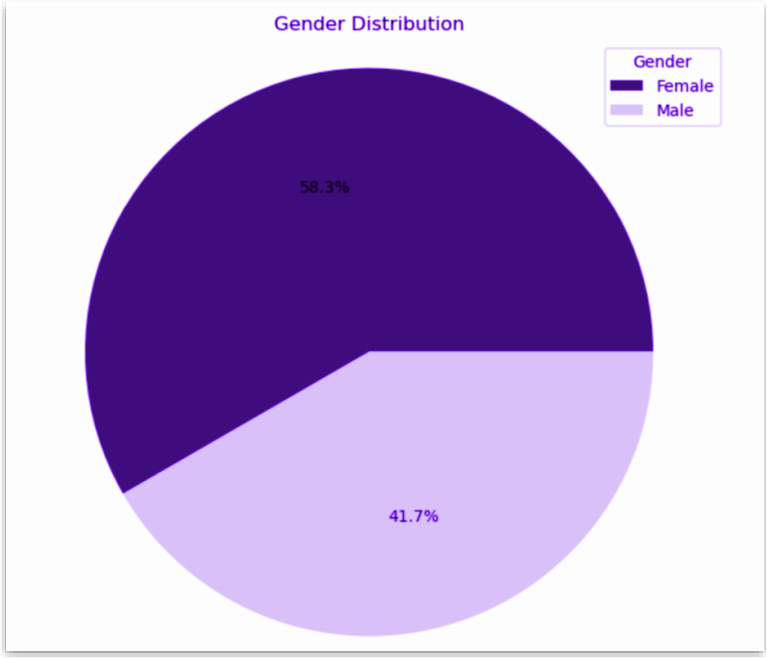
The Dataset

- **Extensive Dataset:** Over 200,000 high-quality images of celebrities.
- **Diverse Attributes:** 40 binary attributes per image, including gender, age, hair color, facial expressions, and more.
- **Large Variations:** Images cover a wide range of poses, backgrounds, and lighting.

Exploratory Data Analysis (EDA)

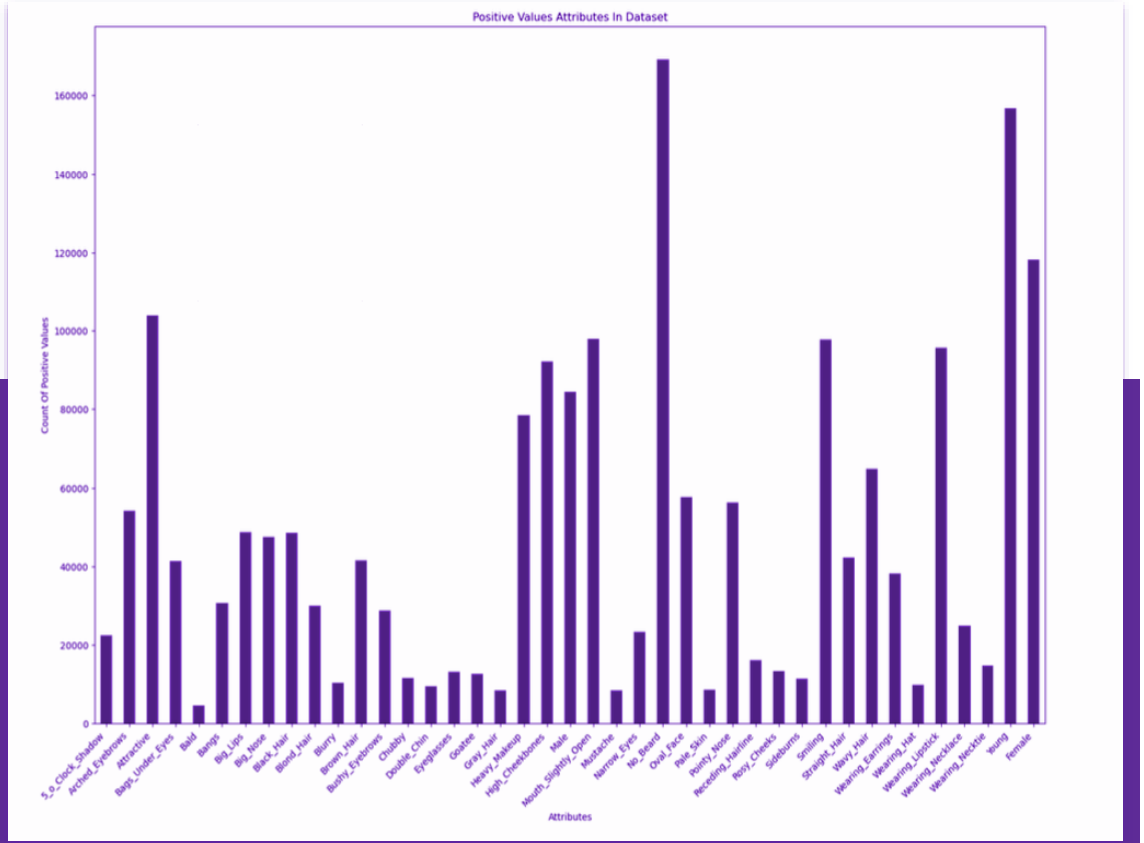
Distribution Of Gender

This indicates that the dataset is slightly skewed towards female individuals



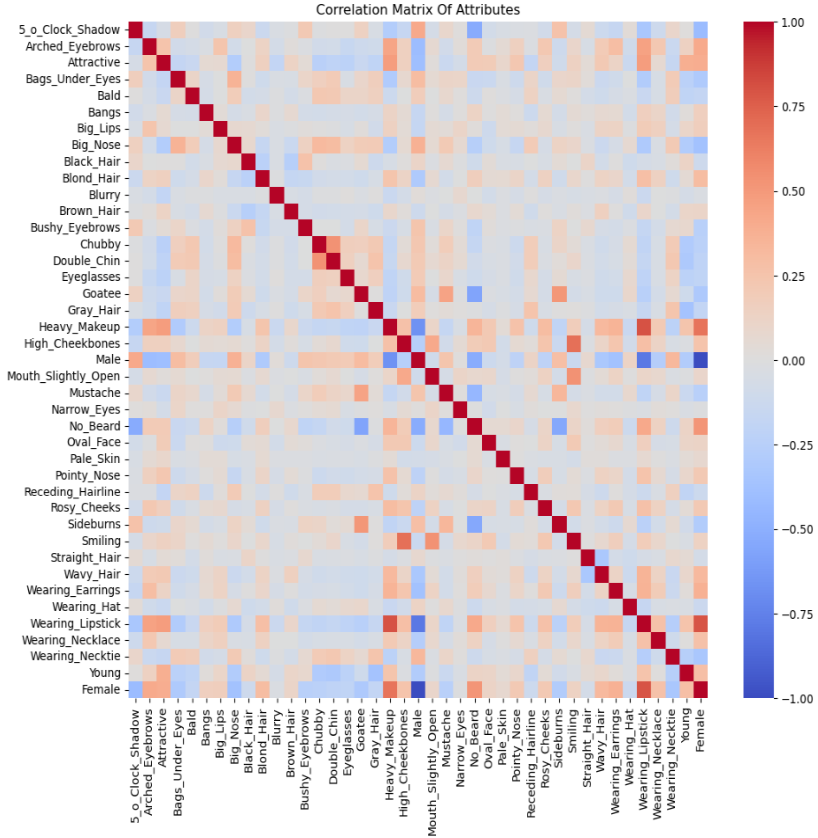
Distribution Of Attributes

- High counts for 'Young' and 'Attractive'
- Highest male attribute count: 'No Beard'



Correlation Matrix Of Attributes

The correlation matrix shows the relationships between various facial attributes

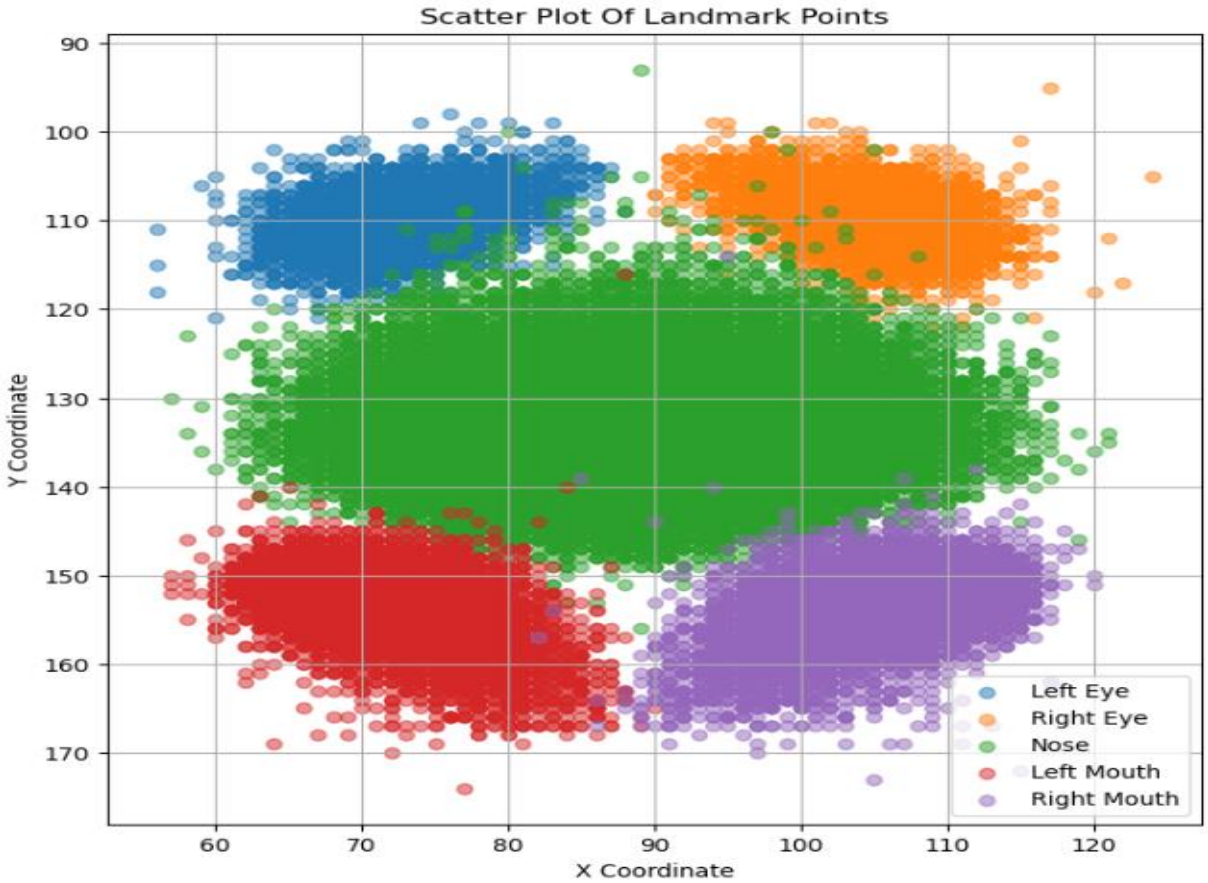


- *Gender:*
  - Strong negative correlation: Male vs. Young/Female
  - Male has a strong positive correlation with No_Beard, indicating that men are less likely to have beards.

- *Facial Hair:*
  - Strong positive correlation: Beard vs. Mustache
  - Sideburns have a moderate positive correlation with Beard, suggesting a connection between these facial hair attributes.

- *Age:*
  - Strong negative correlation: Young vs. Bald /Gray_Hair /Receding_Hairline

- *Appearance:*
  - Moderate positive correlation: Attractive vs. High_Cheekbones


Scatter Plot Of Landmark Points
Y Coordinate
X Coordinate
90
100
110
120
130
140
150
160
170
60
70
80
90
100
110
120
Left Eye
Right Eye
Nose
Left Mouth
Right Mouth

Applications:

- *Face Detection :*

  Training models to accurately identify faces in images.

- *Facial Attribute Recognition :*

  Classifying facial characteristics like gender, hair color, expressions, etc.

- *Image Segmentation :*

  Isolating specific facial regions like eyes or lips.

- *Image Generation :*

  Creating realistics facial images or manipulating existing ones.

Problem Definition

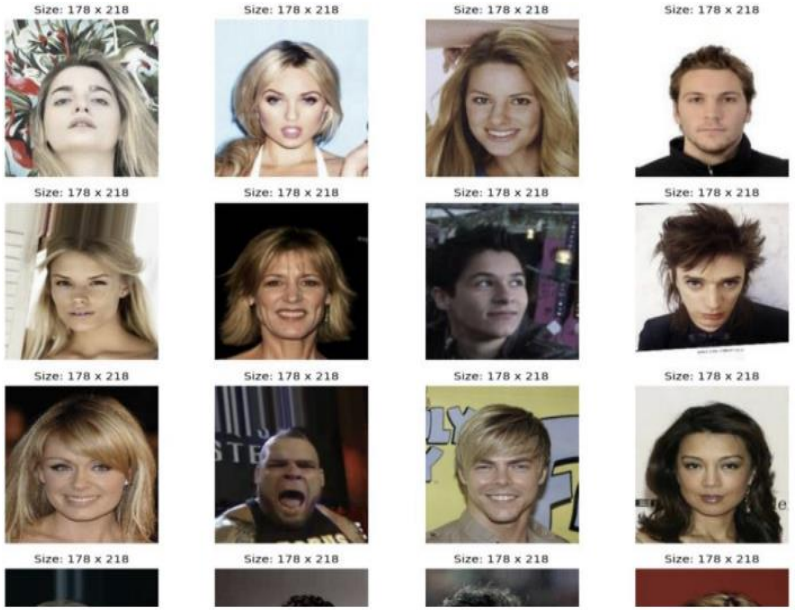
- **Facial attribute recognition using face recognition library and RNN**
- **CelebA Dataset for Gender Detection Using InceptionV3**
- **Large-scale CelebFaces Attributes (CelebA) Dataset to train DCGAN**

Data Preprocessing

- Cleaned the data with normalization and data augmentation
- No missing values

Data Normalization

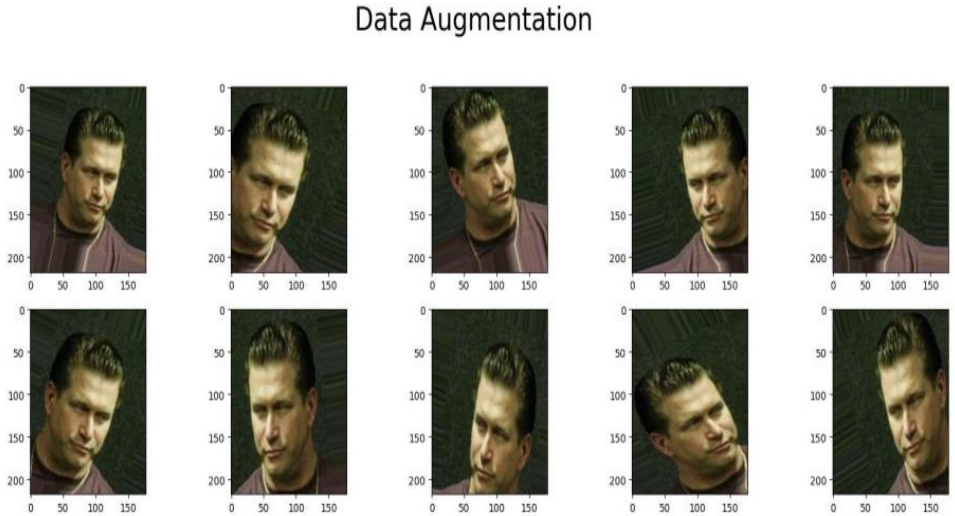
- Normalization is a process that changes the range of pixel intensity values for the images in the database. We are taking 5000 random images and normalizing the images and storing them in another folder.
- Normalization helps in converging the images more consistently.



Data Augmentation

- Data augmentation is a technique used to artificially increase the size and diversity of a dataset by applying various transformations to existing data. The following transformations are applied to the data.

1. Rotation
2. Shifting
3. Zooming
4. Flipping



Machine Learning Models

- Face Recognition Model
- Rectifier Neural Networks
- InceptionV3
- DCGAN

Face Recognition

- Used to recognize facial pointers for eyes, nose, mouth, chin.
- Facial landmarks are detected correctly for 99% of the images we tested on. It's getting detected even when the face is not in proper alignment.

Haar Cascades

- Haar cascades are a machine learning object detection method used to identify objects in images or video streams with the help of Haar features.

- Haar Features: These are simple rectangular features that capture the presence of specific patterns in an image, such as edges and textures.

- Haar cascades use an integral image, which allows for rapid calculation of the sum of pixel values in any rectangular area of the image.

- In our project Haar Cascades are failing when the image is not in proper alignment.

Detecting if the person in the image is smiling, bald, or young and generating images with the respective values from the attributes file

Rectifier Neural Networks (RNN)

- Here we are using rectifier neural networks. Importing Keras, and TensorFlow to train a model to know if the person in the image is bald or not.
- We are using Max Pooling, dropout, density, conv2D, and Relu to train the model.
- We have achieved 97.7% accuracy to train the model and loss is 1.32 percent.

```
accuracy: 0.9777 - loss: 0.1320
```

RNN

- Conv2d: While RNNs are designed for sequential data, you can use Conv2D layers to process images first before passing the features to RNN layers. This is useful because convolutional layers can effectively capture spatial hierarchies in image data.

- Max Pooling: Max Pooling is generally used after Conv2d to reduce the dimensionality of the feature maps which helps in managing complexity and improving performance.

- Dropout: Dropout is a regularization layer and Dropout layers help mitigate overfitting.

- Dense layers provide the final classification output, using **ReLU** for non-linearity in intermediate layers.

RNN

- We are using RNN layers to capture dependencies in the image data, which is structured as sequences of pixel rows.
- After RNN layer we are adding more layer with ReLU activation which allows the model to learn more complex features from the output of RNN layers.
- Finally, add a dense layer with a sigmoid activation function to output the binary output of the bald attribute.
- The use of ReLU in the dense layers helps the model learn rich feature representations.

InceptionV3

- Serve as a deep convolutional neural network for efficient image classification and feature extraction.
- Batch normalization stabilizes and accelerates training.
- Pre-trained weights enhance feature extraction by leveraging learned features from large datasets.

```
Model Evaluation
Test accuracy: 94.4500%
f1_score: 0.9422776911076443
```

Female

91.39% prob.

Real Target: Female

Eyebrows: Not Arched

Bald: No

Young: Yes

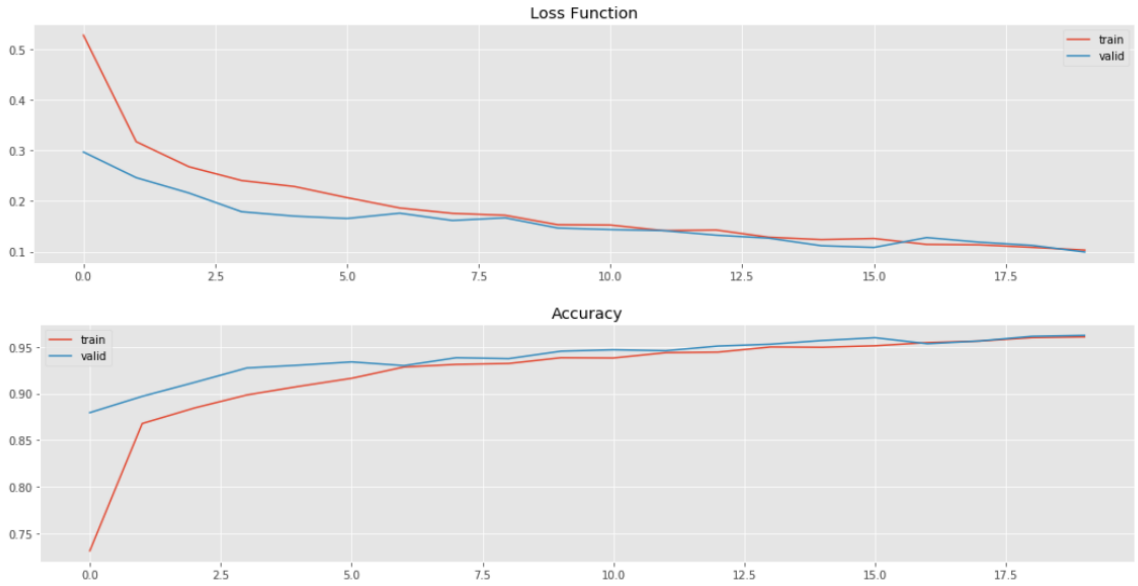
Straight Hair: No

Wavy Hair: No

Smiling: No

Eye Bags: No

Nose: Small

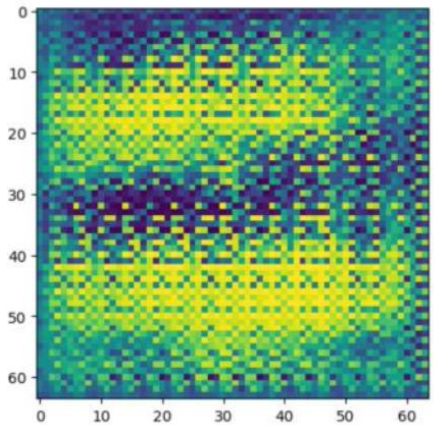


Decreasing Loss: Both the training and validation loss are decreasing, which is a good sign.

DCGAN Architecture

- Generator Architecture
- Discriminator Architecture
- Combining Generator and Discriminator

Generator Architecture



Input: Latent vector (random noise)

Transposed Convolutional Layers: Upsample latent vector

Batch Normalization: Stabilizes training

ReLU Activation: Introduces non-linearity

Output: Tanh activation to produce final image

Example Structure

- Input: 100-dimensional noise vector
- Dense layer: 8192 units, reshaped to (128, 8, 8)
- Transposed Conv Layer 1: 256 filters, 4x4 kernel, stride 2, padding 1
- Transposed Conv Layer 2: 128 filters, 4x4 kernel, stride 2, padding 1
- Transposed Conv Layer 3: 64 filters, 4x4 kernel, stride 2, padding 1
- Output Layer: 3 channels (RGB), Tanh activation

Model: "sequential_17"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 8192)	827,392
reshape_6 (Reshape)	(None, 4, 4, 512)	0
conv2d_transpose_24 (Conv2DTranspose)	(None, 8, 8, 256)	2,097,408
batch_normalization_24 (...ization)	(None, 8, 8, 256)	1,024
leaky_re_lu_30 (LeakyReLU)	(None, 8, 8, 256)	0
conv2d_transpose_25 (Conv2DTranspose)	(None, 16, 16, 128)	524,416
batch_normalization_25 (BatchNormalization)	(None, 16, 16, 128)	512
leaky_re_lu_31 (LeakyReLU)	(None, 16, 16, 128)	0
conv2d_transpose_26 (Conv2DTranspose)	(None, 32, 32, 64)	131,136
batch_normalization_26 (BatchNormalization)	(None, 32, 32, 64)	256
leaky_re_lu_32 (LeakyReLU)	(None, 32, 32, 64)	0
conv2d_transpose_27 (Conv2DTranspose)	(None, 64, 64, 3)	3,075

Toggle output scrolling

Total params: 3,585,219 (13.68 MB)
Trainable params: 3,584,323 (13.67 MB)
Non-trainable params: 896 (3.50 KB)

Discriminator Architecture

Input: Image (real or generated)

Convolutional Layers: Downsample the image

Batch Normalization: Stabilizes training

LeakyReLU Activation: Introduces non-linearity

Output: Sigmoid activation

Example Structure

- Input: Image (64x64x3)
- Conv Layer 1: 64 filters, 4x4 kernel, stride 2, padding 1
- Conv Layer 2: 128 filters, 4x4 kernel, stride 2, padding 1
- Conv Layer 3: 256 filters, 4x4 kernel, stride 2, padding 1
- Conv Layer 4: 512 filters, 4x4 kernel, stride 2, padding 1
- Output Layer: 1 unit, Sigmoid activation

Model: "sequential_18"

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 32, 32, 64)	1,792
leaky_re_lu_33 (LeakyReLU)	(None, 32, 32, 64)	0
dropout_12 (Dropout)	(None, 32, 32, 64)	0
conv2d_13 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_27 (BatchNormalization)	(None, 16, 16, 64)	256
leaky_re_lu_34 (LeakyReLU)	(None, 16, 16, 64)	0
dropout_13 (Dropout)	(None, 16, 16, 64)	0
flatten_6 (Flatten)	(None, 16384)	0
dense_13 (Dense)	(None, 1)	16,385

Total params: 55,361 (216.25 KB)

Trainable params: 55,233 (215.75 KB)

Non-trainable params: 128 (512.00 B)

Combining Generator and Discriminator

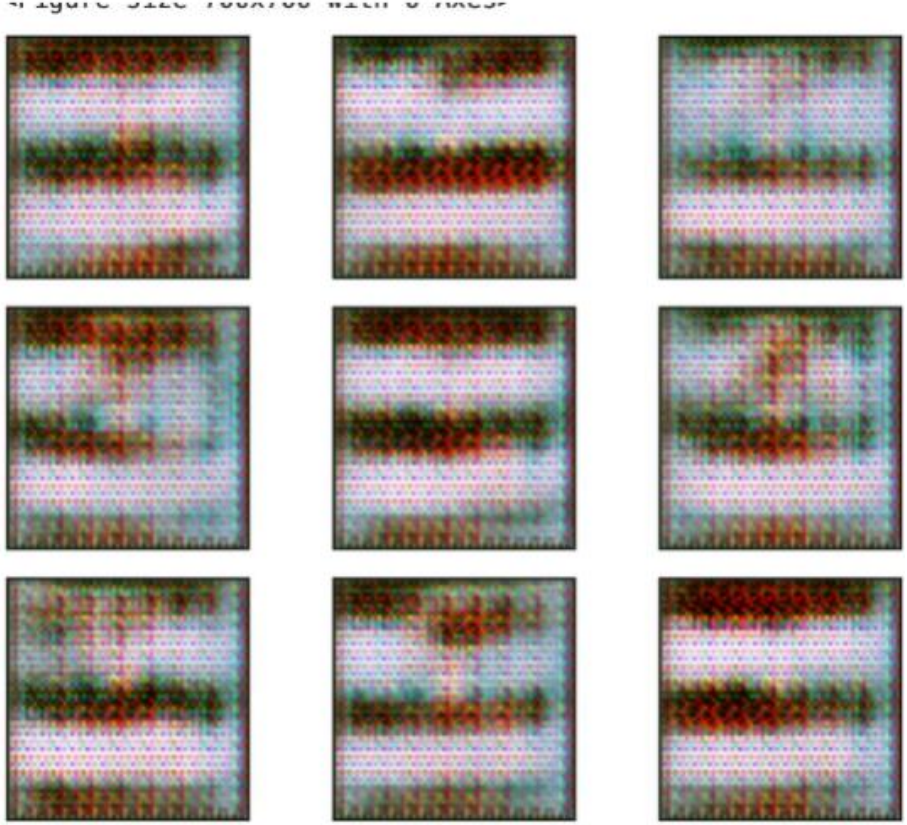
Why would we want to do this?

Why Combine the Generator and Discriminator Models?

- ***Training Synergy***
- ***Adversarial Training***
- ***Enhanced Learning***
- ***Feedback Loop***
- ***Balanced Optimization***
- ***Realistic Outputs***

Results

Predicted Noise



After Training

Conclusion

- We compared our model with haarcascades and facial recognition models is doing a better job in recognizing the facial attributes when compared to haar cascades as facial attributes are not properly recognized with haar cascades when the face is not in proper alignment.

Conclusion

- We compared the facial attribute recognition using facial recognition and RNN to inception V3 model and inception V3 model is doing a better job at attribute recognition because it will work for any image outside dataset and doesn’t need the attribute file when testing the images.

Thank you